

1. a) classify different types of operating systems suitable for this scenario and justify your choice
- Since the OS supports multiple users, multiple processes, CPU, memory, and I/O sharing, the most suitable operating system.

1. Multi-user operating system

Allows multiple users to access the system simultaneously.

Each user has a separate account and permissions.

Ex: linux, UNIX

2. Multitasking operating system

Executes multiple programs at the same time.

CPU switches rapidly between processes.

Improve system efficiency

3. Multiprocessing operating system

uses two or more CPUs/cores

Executes multiple processes simultaneously

Increase performance and reliability



## A. Time-sharing operating system

shares CPU time among many users.

Each user gets a small time slice.

Provides fast response to all users.

A multi-user time sharing operating system with multiprocessing support is most suitable because it efficiently shares CPU, memory, and I/O devices among multiple users and processes.

b) Draw the structure of an operating system showing how resources are managed.

Users



Application programs



system calls/API



Operating system

process management

Memory Management

File Management

I/O Device Manager

security



Hardware Resources

CPU | RAM | Disk |

Keyboard | printer



## System call sequence

start

|  
fork() → create a child process

|  
child process

|  
exec() → load and execute new program

|  
exit() → child process terminates

|  
parent process

|  
wait() → wait for child completion

|  
End



2. a) categorize system calls used for file and device management.

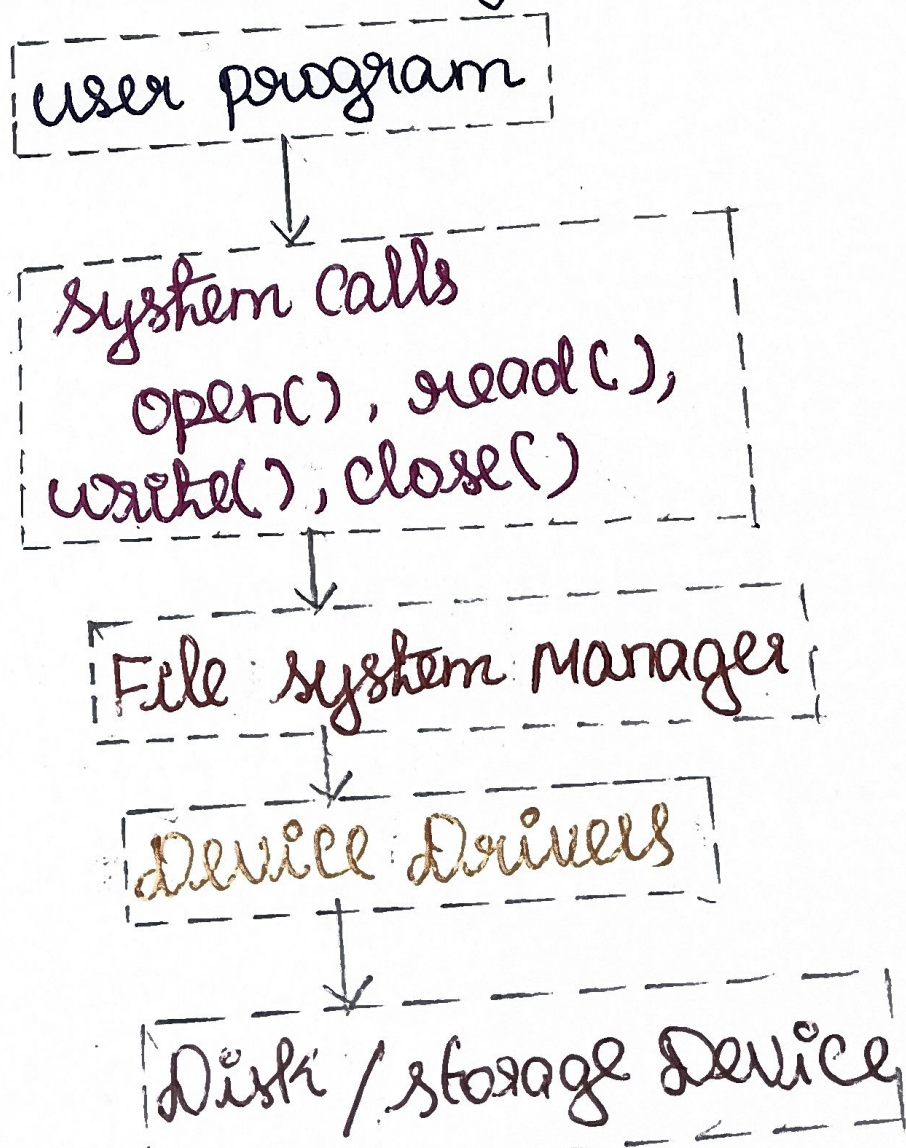
File management system calls

create()  
open()  
read()  
write()  
close()  
delete()

Device management system calls

request()  
release()  
read()  
write()  
ioctl()

b) layered structure diagram for file operations





2. a) C) C-based system call sequence to open, read and close a file.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;
    char buffer[100];
    fd = open("sample.txt", O_RDONLY);
    if (fd >= 0)
    {
        read(fd, buffer, sizeof(buffer));
        printf("%s", buffer);
        close(fd);
    }
    return 0;
}
```

system call sequence

program starts

↓  
open("sample.txt")

↓  
read(fd, buffer, size)

↓  
process data

↓  
close(fd)

↓  
program ends